Table of Contents
Chapters 1–4

Randomized Reward Shaping Deep Reinforcement Learning
for Self-Adaptive Systems

Alireza Kavianifar

February 15, 2026

Contents

1.1 Background and Context

1.1.1 Self-Adaptive Systems: motivation and examples

1.1.2 The MAPE-K feedback loop and the Managed/Managing System abstraction

1.1.3 Limitations of rule-based and static adaptation

1.2 Runtime Uncertainty in Self-Adaptive Systems

1.2.1 Sources of uncertainty (environmental, system-level)

1.2.2 Impact of uncertainty on adaptation decisions (examples from DeltaIoT)

1.3 Learning-based Adaptation: Why Reinforcement Learning?

1.3.1 RL benefits for runtime decision-making

1.3.2 Limits of classic RL in large adaptation spaces

1.4 What "Continuous" Means in This Thesis

1.4.1 Continuous = continuous runtime learning / streaming adaptation cycles (not continuous-action control)

1.5 Problem Statement (precise formulation extracted from the article)

1.6 Proposed Solution Overview: RS-DRL (high-level)

1.6.1 Key idea: randomized reward reshaping during experience replay

1.6.2 Architectural placement within MAPE-K (on-demand retraining & async learning)

1.7 Research Objectives and Research Questions

1.8 Contributions of the Thesis (expanded from the article's contribution list)

1.9 Research Methodology Overview

1.9.1 Theoretical modeling and RL formulation

1.9.2 Algorithm design and reward shaping experiments

1.9.3 Case study (DeltaIoT): datasets, training regime, evaluation metrics

1.10 Thesis Organization (brief outline of subsequent chapters)

2.1 Self-Adaptive Systems: Concepts and Architectural Patterns

2.1.1 MAPE-K detailed (Monitor, Analyze, Plan, Execute, Knowledge)

2.1.2 Managed vs Managing System abstraction and temporal classification (monitoring vs asynchronous learning)

2.2 Uncertainty in Self-Adaptive Systems

2.2.1 Taxonomy of uncertainties (environmental, systemic, goal drift)

2.2.2 Effects of uncertainty on verification and adaptation (motivating evidence from DeltaIoT)

2.3 Reinforcement Learning Foundations for SAS

2.3.1 MDPs, policy/value formulation and why RL suits adaptation problems

2.3.2 Known challenges: local optima, generalization, retraining cost

2.4 Deep Reinforcement Learning in Large Discrete Adaptation Spaces

2.4.1 DQN architecture and rationale for discrete adaptation options (why DQN was chosen in the article)

2.4.2 Experience replay, target networks, stability techniques used

2.5 Reward Shaping and Experience Relabeling (state of the art)

2.5.1 HER and goal-conditioned methods — contrast to RS-DRL (no pre-defined goals)

2.5.2 Other generalization techniques

2.6 Related Work: classification and gaps

2.6.1 ML-based methods, search-based planners, RL-based adaptation — strengths and weaknesses (article's survey)

2.6.2 Identified gap: scalable, uncertainty-aware, continuous runtime DRL for large adaptation spaces

2.7 Chapter Summary (lead into Chapter 3's formal modeling)

3.1 Motivating Case Study: DeltaIoT System (detailed description)

3.1.1 Network, motes, links, configuration parameters and examples

3.2 Modeling Uncertainty in DeltaIoT and SAS

3.2.1 Interference/noise ranges, message-load variability, data shifts (Fig.2 evidence)

3.2.2 How uncertainty affects QoS metrics (packet loss, latency, energy)

3.3 Adaptation Space and Configuration Representation

3.3.1 Definition: adaptation option, parameter encoding, scale (216 / 1,296 / 4,096)

3.3.2 Implications for search/learning complexity

3.4 RL Formalization for the Adaptive Problem (exact equations from paper)

3.4.1 State space definition (e.g., $S = (E, P, L)$)

3.4.2 Action space: discrete adaptation options and encoding

3.4.3 Reward functions: multi-objective normalization, threshold/setpoint formulations (include equations 11–12 and Table 2)

3.4.4 Transition dynamics (how verifier / ActivFORMS produces next states)

3.5 Continuous / Streaming Training Assumptions

3.5.1 Treating adaptation cycles as a continuous stream (training horizons and validation splits per instance)

3.5.2 Temporal separation: runtime adaptation vs asynchronous learning (design assumption)

3.6 Verification & Runtime Models for Option Analysis

3.6.1 UPPAAL-SMC models and ActivFORMS execution engine (role in reward calculation)

3.7 Formal Problem Statement (thesis version) — precise objective and constraints (ready to feed Chapter 4)